

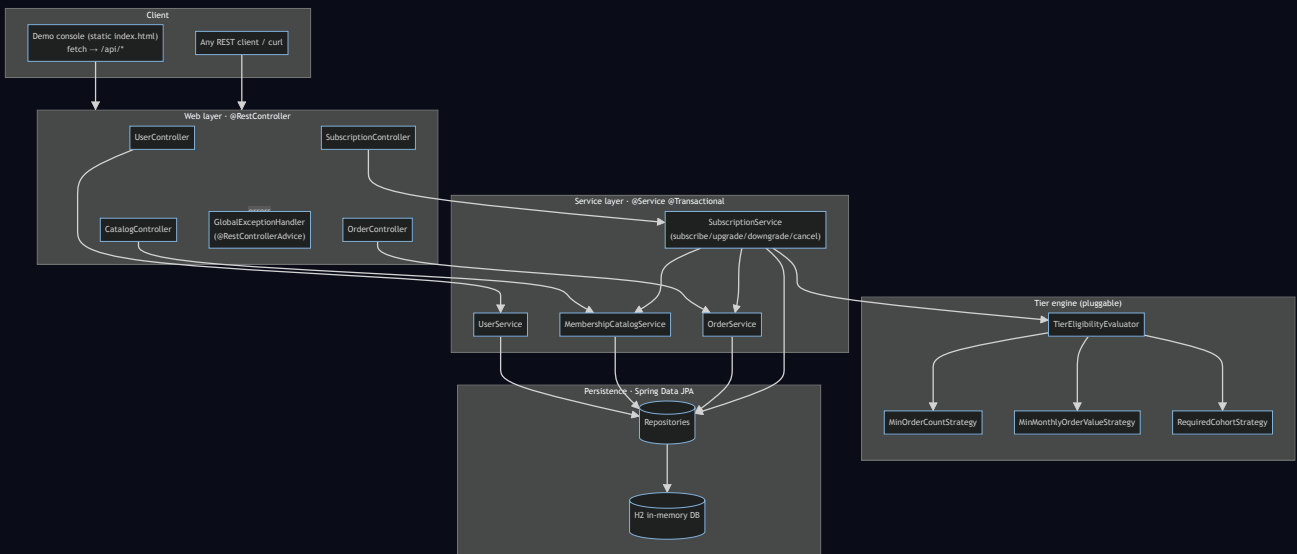
Architecture & Flow — FirstClub Membership Program

Java 21 · Spring Boot 3.4 · layered design · Strategy-based tier engine · optimistic-locking concurrency

[➤ Demo console](#) [➤ Entity design](#) [Components](#) [Request flow](#) [Subscribe flow](#) [Tier engine](#) [Concurrency](#)

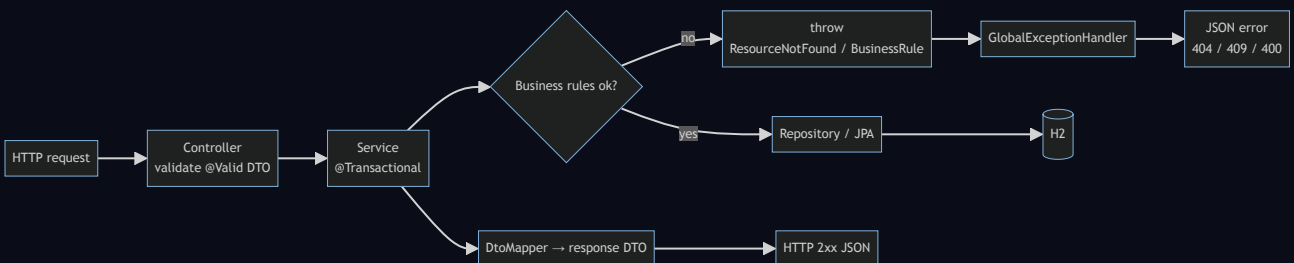
1 · Component architecture

Classic layered architecture. Each layer depends only on the one below it; the tier eligibility engine is an isolated, pluggable module.



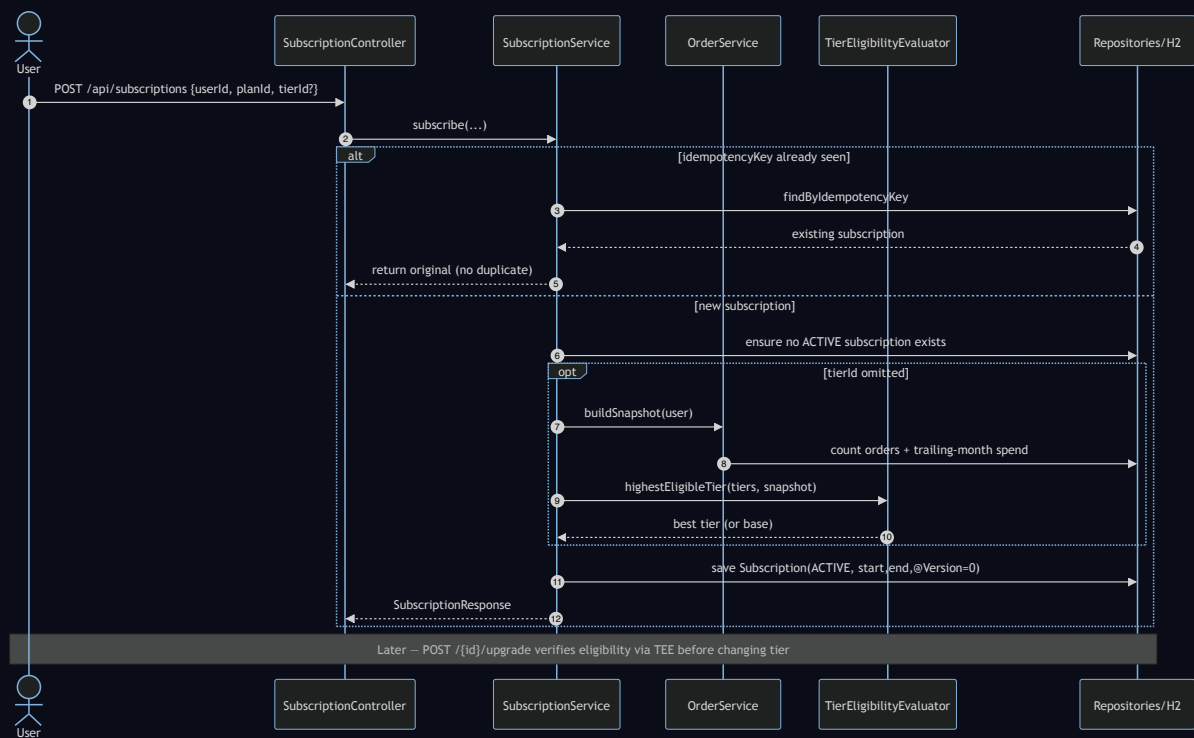
2 · Request flow (high level)

Thin controllers translate DTOs $\rightleftharpoons$ domain; services own the transaction boundary and business rules.



3 · Subscribe & tier-promotion flow

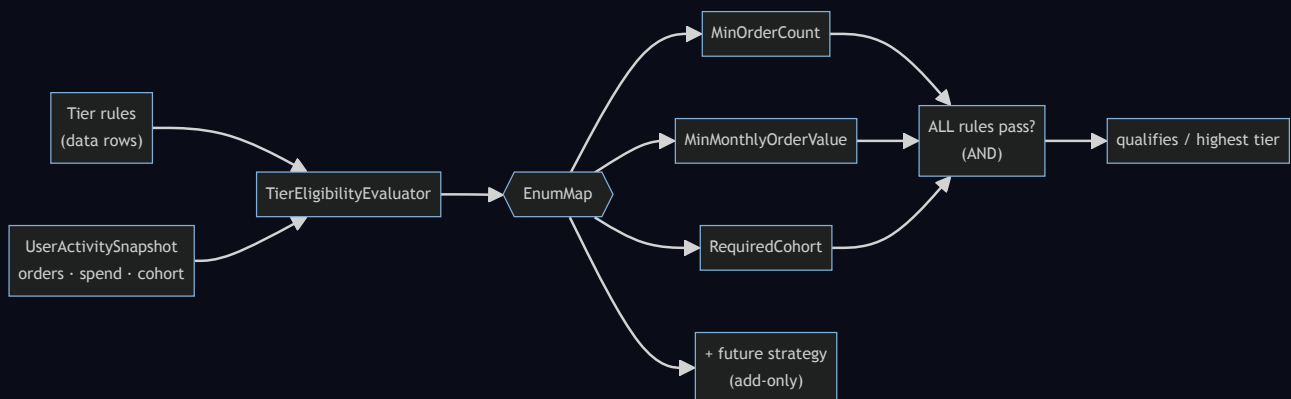
Subscribing with no tier auto-assigns the highest tier the user qualifies for. Upgrades enforce eligibility.



4 · Extensible tier engine (Strategy + Open/Closed)

Tier promotion is data-driven, not hard-coded. Each criterion is its own strategy bean.

- **TierEligibilityStrategy** — interface with `supportedType()` + `isSatisfied(rule, snapshot)`.
- **Implementations:** `MinOrderCountStrategy`, `MinMonthlyOrderValueStrategy`, `RequiredCohortStrategy`.
- **TierEligibilityEvaluator** injects all strategies into an `EnumMap` and applies a tier's rules with **AND** semantics.
- **Adding a new criterion** = add an `EligibilityRuleType` + one bean. No existing class changes (Open/Closed).
- **Configurable at runtime:** benefits (`TierBenefit`) and rules (`TierEligibilityRule`) are rows, editable via admin APIs.

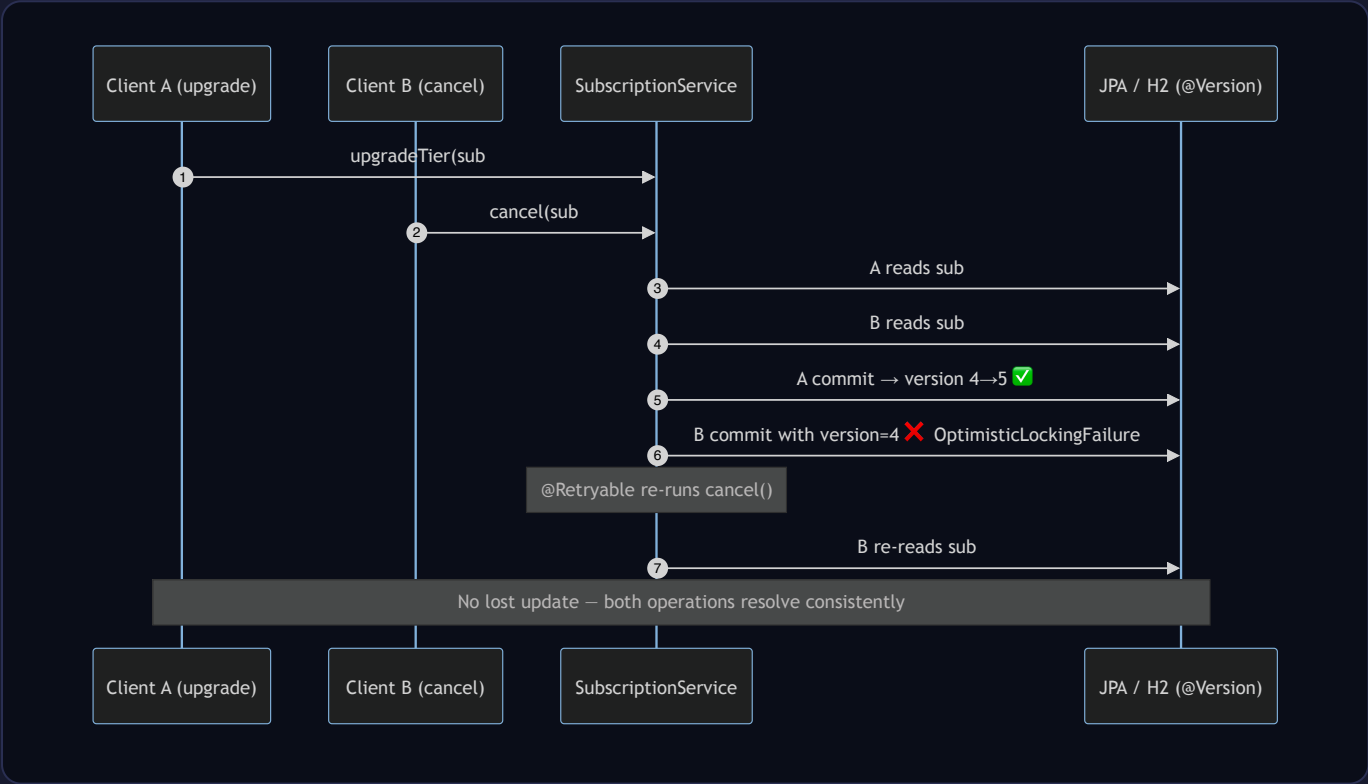


5 · Concurrency approach & design

The bonus ask. Concurrent mutations on the same subscription must never silently clobber each other.

Mechanism	Where	Protects against
Optimistic locking (<code>@Version</code>)	<code>Subscription.version</code>	Lost updates from concurrent upgrade/downgrade/cancel
Automatic retry (<code>@Retryable</code> on <code>OptimisticLockingFailureException</code>)	<code>SubscriptionService</code> mutators	Transient version clashes — retried up to 3× with backoff
Idempotency key (unique)	<code>subscribe(...)</code>	Duplicate subscriptions from retries / double-clicks
Single-active invariant	<code>SubscriptionService</code>	Two active subscriptions per user
Injectable <code>Clock</code>	order/expiry logic	Flaky time-based behaviour; enables deterministic tests

Sequence: two clients mutate the same subscription concurrently.



Proven by `SubscriptionConcurrencyTest` — N threads upgrade/downgrade the same subscription in parallel; the final state is consistent and no update is lost.